

PricePilot AI

Title: PricePilot AI: Dynamic Pricing Optimization & Revenue Intelligence System
Milestone-2 (Machine Learning, Price Intelligence, Backend & Database Implementation)

Project Objective

The objective of **PricePilot AI** is to build an AI-powered dynamic pricing and revenue intelligence platform that helps businesses make data-driven pricing decisions using demand patterns, product information, inventory conditions, competitor pricing, promotions, and historical sales data.

Following the successful completion of **Milestone 1 (EDA part)**, Milestone 2 focused on transforming the prepared dataset and system architecture into an operational **Machine Learning and backend intelligence layer**.

The major objectives of Milestone 2 were:

- Develop demand prediction models
- Compare multiple Machine Learning algorithms
- Perform model evaluation and optimization
- Select the best-performing model
- Experiment with deep learning using LSTM
- Perform time-series feature experiments
- Build the backend API architecture
- Configure PostgreSQL database
- Implement database migrations using Alembic
- Implement authentication and authorization
- Implement product, inventory and competitor management APIs
- Implement pricing and recommendation APIs
- Integrate AI-powered endpoints
- Prepare the system for ML inference and pricing intelligence

The Milestone 2 implementation follows the project requirement of developing **price prediction, demand forecasting, pricing recommendation and AI-powered pricing intelligence workflows**.

Step 1: Machine Learning Environment Setup

The Machine Learning environment was configured using Python and the required data science and ML libraries.

Technologies Used

- Python
- Pandas
- NumPy
- Matplotlib
- Seaborn
- Scikit-learn
- XGBoost
- TensorFlow
- Keras
- Google Colab
- VS Code

The ML pipeline was developed separately from the application backend so that the trained model, preprocessing pipeline and inference layer can later be integrated into the FastAPI application.

Step 2: Dataset Preparation for Machine Learning

The cleaned dataset prepared during Milestone 1 was used for Machine Learning development.

The dataset contained:

- **76,000 records**
- **25 engineered columns after preprocessing and feature engineering**
- Historical sales information
- Product information
- Pricing information
- Competitor pricing
- Inventory information
- Promotion information
- Seasonal information
- Epidemic indicator
- Date-based features

The target variable selected for the main ML task was:

Target Variable: Demand

The reason for selecting **Demand** as the primary ML target was that demand prediction provides the foundation for pricing intelligence.

The system can first estimate expected demand and subsequently use that prediction within pricing and revenue optimization workflows.

Step 3: Feature Engineering

Additional features were created to improve the ability of the models to understand pricing, competitive and temporal patterns.

Date Features

```
df['Year'] = df['Date'].dt.year
```

```
df['Month'] = df['Date'].dt.month
```

```
df['Day'] = df['Date'].dt.day
```

A

Additional temporal features were created:

- Day of Week
- Quarter
- Week of Year
- Month Sin
- Month Cos
- Day of Week Sin
- Day of Week Cos

These features allow the model to capture periodic and seasonal patterns.

Competitive Pricing Features

A major feature for the PricePilot AI system was the difference between the company's price and competitor pricing.

```
df['Price Difference'] = (  
    df['Price'] - df['Competitor Pricing']  
)
```

Additional competitive features included:

- Price Difference
- Relative Price Difference
- Relative Price Difference %
- Price Position

The **Price Position** feature helps identify whether the company's product is:

- Cheaper
- Similar
- More expensive

These features are particularly relevant to the project's competitive pricing intelligence requirements.

Step 4: Machine Learning Feature Selection

The final ML feature set contained **26 features**.

Categorical Features

- Store ID
- Product ID
- Category
- Region
- Weather Condition
- Seasonality
- Price Position

Numerical Features

- Inventory Level
- Units Ordered
- Price
- Discount
- Promotion
- Competitor Pricing
- Epidemic
- Year
- Month
- Day
- Price Difference
- Relative Price Difference
- Day of Week
- Quarter
- Week of Year
- Month Sin
- Month Cos
- DayOfWeek Sin
- DayOfWeek Cos

Excluded Variables

The following variables were excluded from the main prediction features:

- Date
- Demand
- Units Sold
- Revenue
- Inventory Group
- Relative Price Difference %

Units Sold and Revenue were excluded because they represent outcomes associated with the observed demand/sales period and could introduce target leakage when used for forecasting.

Step 5: Preprocessing Pipeline

A Scikit-learn preprocessing pipeline was implemented.

Numerical Data

Numerical variables were standardized using:

StandardScaler()
Categorical Data

Categorical variables were converted using:
OneHotEncoder(
 handle_unknown='ignore'
)

ColumnTransformer

The preprocessing process was combined using ColumnTransformer.
The resulting pipeline provides consistent transformations between:

- Training
- Validation
- Testing
- Future inference

This is important because the same preprocessing logic must be applied when the model is eventually called through the backend API.

Step 6: Chronological Train, Validation and Test Split

Since the project deals with historical business data, a chronological split was used instead of random splitting.

Dataset Split

Dataset	Records	Time Period
Training	53,200	2022-01-01 → 2023-06-16
Validation	11,400	2023-06-17 → 2023-10-08
Testing	11,400	2023-10-09 → 2024-01-30

This prevents future observations from being randomly mixed into the training data.
The test dataset was kept as a held-out evaluation period.

Step 7: Machine Learning Models

Multiple regression models were evaluated to identify the most suitable model for demand prediction.
The following models were tested:

1. Linear Regression
2. Ridge Regression
3. Lasso Regression
4. Decision Tree Regressor
5. Optimized Decision Tree
6. Random Forest Regressor
7. XGBoost Regressor
8. LSTM Neural Network

This model progression allowed both simple linear models and advanced nonlinear models to be compared.

Step 8: Linear Regression

Linear Regression was implemented as the initial baseline model.

Validation Results

Metric	Result
MAE	25.59
RMSE	33.39
R ²	0.5201
MAPE	31.48%

The model explained approximately **52.01% of the variance in demand**.

The results showed that a simple linear relationship was not sufficient to capture all the complex relationships present in the dataset.
Therefore, nonlinear models were investigated.

Step 9: Ridge and Lasso Regression

Regularized linear models were evaluated to determine whether controlling model complexity could improve the baseline.

Ridge Regression

Metric	Result
MAE	25.22
RMSE	32.91
R ²	0.5337
MAPE	30.73%

Lasso Regression

Metric	Result
MAE	25.22
RMSE	32.91
R ²	0.5337
MAPE	30.73%

Both models provided a small improvement over standard Linear Regression.
However, the improvement was limited, indicating that regularization alone could not capture the nonlinear relationships in demand.

Step 10: Decision Tree Regression

A Decision Tree Regressor was implemented to capture nonlinear feature interactions.

Initial Validation Results

Metric	Result
MAE	21.87
RMSE	32.87
R ²	0.5350
MAPE	23.92%

The initial model showed strong training performance but significant overfitting.
Therefore, tree parameters were optimized.

Optimized Decision Tree

Metric	Result
MAE	21.46
RMSE	29.91
R ²	0.6149
MAPE	25.69%

The optimized tree reduced overfitting and improved generalization.

Step 11: Random Forest Regression

Random Forest was then evaluated as an ensemble tree-based model.

Validation Results

Metric	Result
MAE	20.32
RMSE	27.83
R ²	0.6666
MAPE	25.18%

The Random Forest model substantially improved upon the linear and single-tree models.

A controlled GridSearchCV experiment was also performed to identify better hyperparameters.

The selected configuration included:

`n_estimators = 200`

`max_depth = None`

`max_features = sqrt`

`min_samples_split = 2`

`min_samples_leaf = 1`

Random Forest provided a strong baseline for nonlinear demand prediction, but XGBoost performed better.

Step 12: XGBoost Regression

XGBoost was selected as the primary advanced Machine Learning algorithm because the dataset contains complex nonlinear relationships between:

- Price
- Demand
- Competitor pricing
- Product
- Store
- Category
- Promotion
- Inventory
- Seasonality

XGBoost is capable of learning these nonlinear relationships and feature interactions.

Step 13: XGBoost Hyperparameter Optimization

Several controlled configurations were tested.

The best validation configuration was:

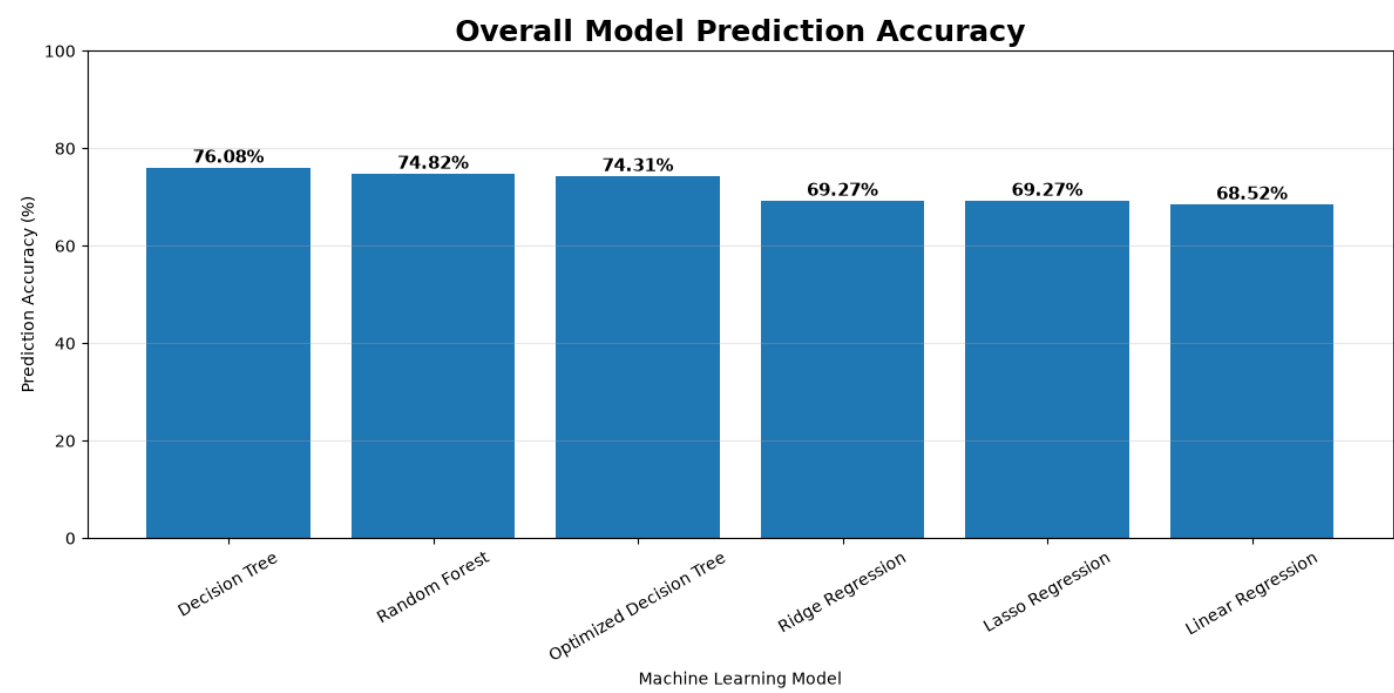
```
XGBRegressor(  
    n_estimators=500,  
    learning_rate=0.05,  
    max_depth=6,  
    subsample=0.8,  
    colsample_bytree=0.8,  
    objective="reg:squarederror",  
    random_state=42
```

)

Best Validation Performance

Metric	Result
MAE	14.01
RMSE	19.96
R ²	0.8285
MAPE	17.45%

The model was selected using validation performance before final evaluation on the untouched test period.



Step 14: Final XGBoost Model Evaluation

After model selection, the final XGBoost model was evaluated on the held-out test dataset.

Final Results

Dataset	MAE	RMSE	R ²	MAPE
Training	11.61	16.15	0.8809	16.19%
Validation	14.01	19.96	0.8285	17.45%
Test	12.43	17.24	0.8527	18.94%

The final model achieved:

$R^2 = 0.8527$

on the held-out test dataset.

This means that approximately **85.27% of the variance in the target demand was explained under the evaluation setup.**

It should not be interpreted as 85.27% individual prediction accuracy.

Step 15: Model Comparison

The major model development results are summarized below.

Model	MAE	RMSE	R ²	MAPE
Linear Regression	25.59	33.39	0.5201	31.48%
Ridge Regression	25.22	32.91	0.5337	30.73%
Lasso Regression	25.22	32.91	0.5337	30.73%
Decision Tree	21.87	32.87	0.5350	23.92%
Optimized Decision Tree	21.46	29.91	0.6149	25.69%
Random Forest	20.32	27.83	0.6666	25.18%
Original XGBoost	16.23	22.69	0.7784	20.27%
Tuned XGBoost	14.01	19.96	0.8285	17.45%
Final XGBoost Test	12.43	17.24	0.8527	18.94%

The project documentation records XGBoost as the current best-performing model.

Step 16: LSTM Deep Learning Experiment

LSTM was evaluated because the project specification identifies LSTM as one of the recommended deep-learning approaches for demand forecasting.

Two LSTM experiments were performed.

Initial LSTM

The initial model used a 30-day sequence containing:

- Demand
- Price
- Discount
- Inventory Level
- Units Ordered
- Competitor Pricing
- Promotion
- Epidemic
- Month
- Day of Week
- Month Sin
- Month Cos

Results

Dataset	MAE	RMSE	R ²	MAPE
Train	29.78	38.61	0.3246	42.22%
Validation	32.46	42.22	0.2328	39.99%
Test	29.15	37.66	0.2969	44.69%

Step 17: Improved LSTM

A more advanced LSTM architecture was developed using:

- Full engineered feature set
- One-hot encoded features
- Store embeddings
- Product embeddings
- Bidirectional LSTM
- Layer Normalization
- Dropout
- Dense layers
- Huber Loss
- Adam optimizer
- Learning-rate reduction
- Early stopping

The sequence size was:

30×39

The model contained approximately:

98,185 trainable parameters

Results

Dataset	MAE	RMSE	R ²	MAPE
Train	28.91	38.10	0.3424	37.95%
Validation	32.76	43.24	0.1951	37.67%
Test	29.95	38.36	0.2705	47.25%

The tested LSTM architectures did not outperform XGBoost on the current dataset representation. The primary issue was predictive underperformance rather than severe overfitting.

Step 18: Lag and Rolling Feature Experiment

Additional time-series features were tested to determine whether historical demand information could improve the XGBoost model.

The following features were introduced:

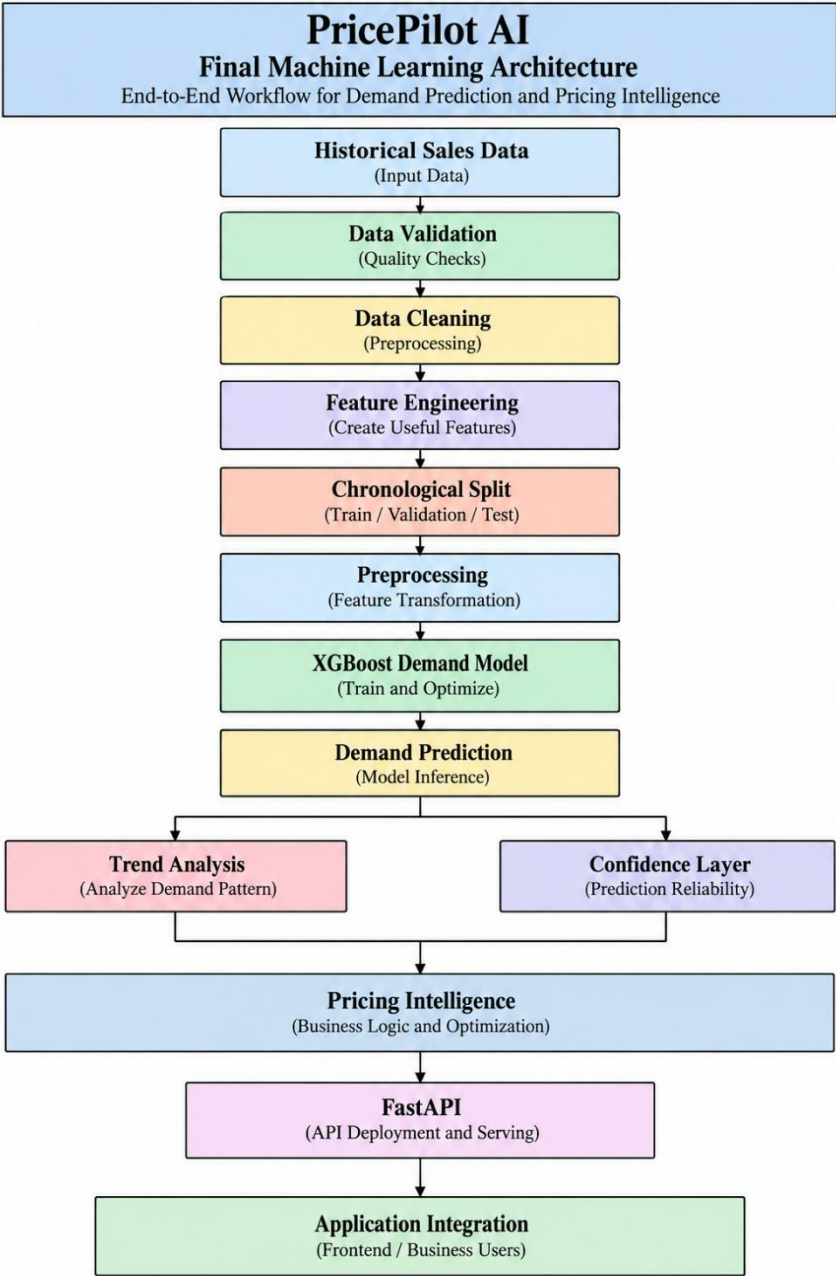
- Demand Lag 1
- Demand Lag 7
- Demand Lag 14
- Demand Lag 30
- Rolling Mean 7
- Rolling Mean 14
- Rolling Mean 30

Shifted historical values were used to avoid directly exposing the current target to the model.

Validation Results

Metric	Result
MAE	16.68
RMSE	22.93
R²	0.7761
MAPE	21.40%

The experiment performed slightly worse than the selected XGBoost configuration and was therefore not adopted as the final model.



This follows the intended forecasting architecture defined for PricePilot AI.

Step 20: Backend Environment Setup

The backend was implemented using **FastAPI**.

The backend provides the application layer through which the frontend can communicate with:

- Authentication
 - Users
 - Organizations
 - Products
 - Categories
 - Inventory
 - Competitors
 - Pricing
 - Sales
 - AI services
-

Step 21: PostgreSQL Database Setup

PostgreSQL was configured as the primary relational database for PricePilot AI.

The database is used to persist:

- User information
- Organizations
- Roles
- Products
- Categories
- Inventory
- Competitors
- Competitor products
- Competitor prices
- Pricing history
- Pricing recommendations
- Sales records
- Authentication information

The database architecture follows a normalized and multi-tenant design.

Step 23: Database Tables Implemented

The PricePilot AI database contains the major entities required for the application.

Authentication and Organization

- organizations
- roles
- users
- user_roles
- organization_members
- refresh_tokens

Product Management

- categories
- products
- product_variants
- inventory

Competitor Intelligence

- competitors

- competitor_products
- competitor_prices

Pricing Intelligence

- price_histories
- pricing_recommendations

Sales

- sales_records

These entities form the data foundation for the pricing intelligence system.

The broader architecture separates organizational data, product information, pricing, sales, inventory and competitor information into related entities.

Step 24: Database Relationship Architecture

The primary logical relationship is:

Organization



This structure allows product-level business intelligence to be connected with pricing, inventory, sales and competitor information.

The project's ER design follows normalized relational principles and supports multi-user, multi-tenant operation.

Step 25: Authentication and Authorization

A secure authentication system was implemented using:

- Password hashing
- JWT authentication
- Access tokens
- Refresh tokens
- Token expiration
- Role-based authorization
- Organization-level authorization

Plaintext passwords and secrets are not intended to be stored in the database.

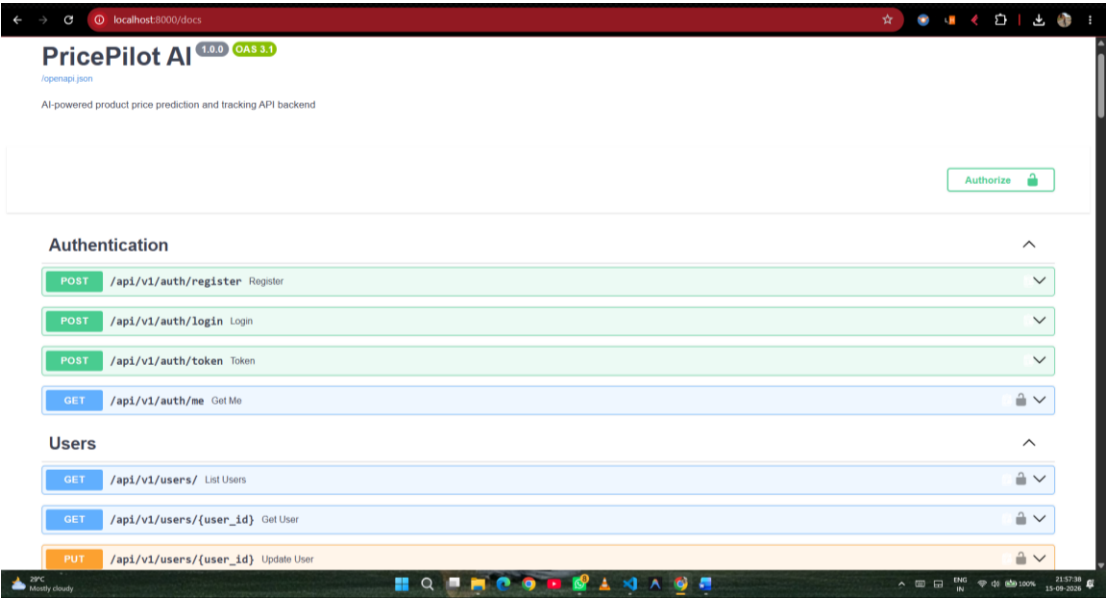
The architecture specification explicitly requires password hashing, JWT access tokens, refresh tokens, role-based authorization and organization-level authorization.

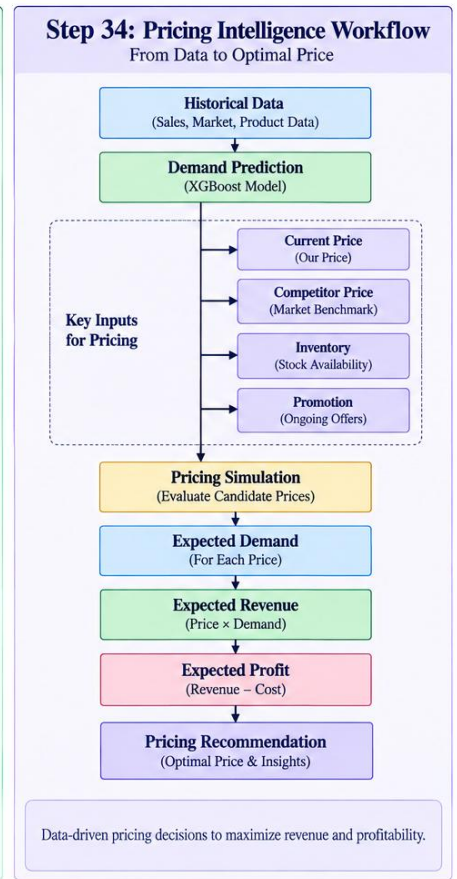
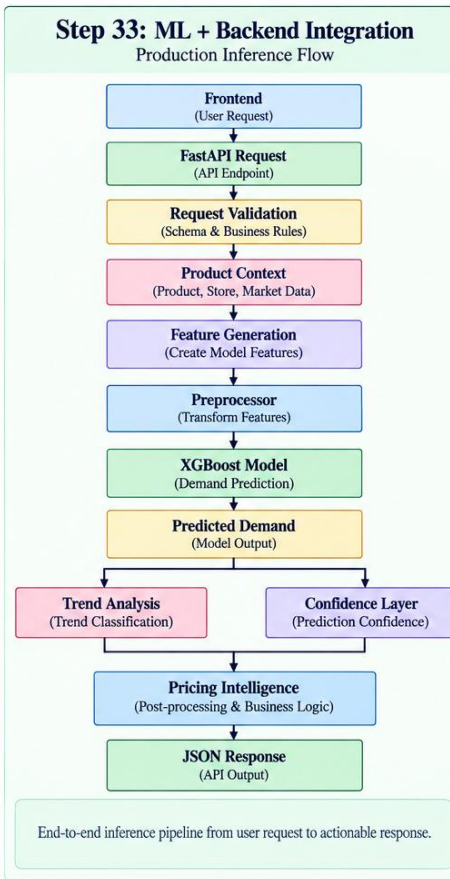
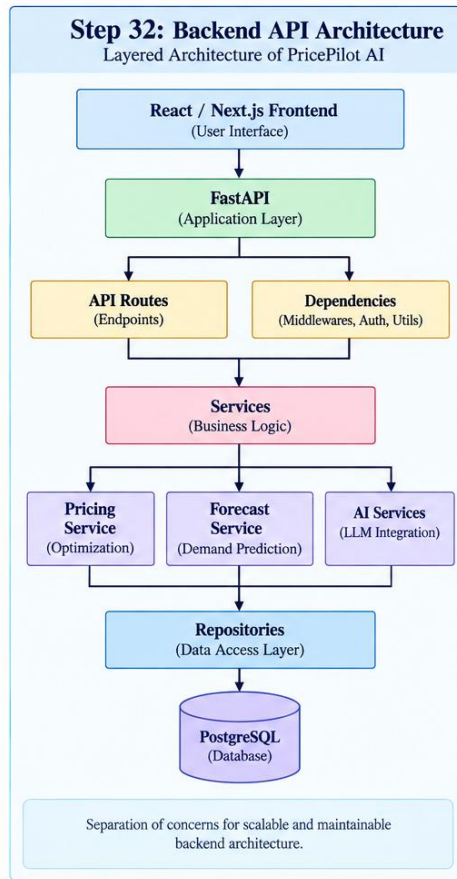
Step 26: Backend API Development

PricePilot AI

Backend API Endpoints Overview

#	Module	Method	Endpoint	Primary Purpose
1	Authentication	POST	/api/v1/auth/register	Register user
2		POST	/api/v1/auth/login	Login
3		POST	/api/v1/auth/token	Generate authentication token
4		GET	/api/v1/auth/me	Get current user
5	Users	GET	/api/v1/users/	List users
6		GET	/api/v1/users/{user_id}	Get user
7		PUT	/api/v1/users/{user_id}	Update user
8	Organizations	GET	/api/v1/organizations/	Get user's organizations
9		POST	/api/v1/organizations/	Create organization
10		GET	/api/v1/organizations/{org_id}	Get organization
11		POST	/api/v1/organizations/{org_id}/members	Add member
12	Categories	POST	/api/v1/categories/	Create category
13		GET	/api/v1/categories/organization/{org_id}	List categories
14	Products	POST	/api/v1/products/	Create product
15		GET	/api/v1/products/organization/{org_id}	List products
16		GET	/api/v1/products/{product_id}	Get product
17		PUT	/api/v1/products/{product_id}	Update product
18		DELETE	/api/v1/products/{product_id}	Delete product
19		POST	/api/v1/products/{product_id}/variants	Add variant
20		GET	/api/v1/products/{product_id}/inventory	Get inventory
21		PUT	/api/v1/products/{product_id}/inventory	Update inventory
22	Competitors	POST	/api/v1/competitors/	Add competitor
23		GET	/api/v1/competitors/organization/{org_id}	List competitors
24		PUT	/api/v1/competitors/{competitor_id}	Update competitor
25		POST	/api/v1/competitors/match	Match competitor product
26		POST	/api/v1/competitors/prices	Record competitor price
27		GET	/api/v1/competitors/product/{product_id}/prices	Get competitor prices
28	Pricing Engine	POST	/api/v1/pricing/predict	Predict pricing outcome
29		POST	/api/v1/pricing/recommendations	Create pricing recommendation
30		POST	/api/v1/pricing/recommendations/{recommendation_id}/apply	Apply recommendation
31		GET	/api/v1/pricing/history/{product_id}	Get price history
32	Sales	POST	/api/v1/sales/	Record sale
33		GET	/api/v1/sales/analytics/{organization_id}	Sales analytics
34	AI v2	POST	/api/v2/ai/gemini	Gemini AI
35		POST	/api/v2/ai/grok	Grok AI
36		POST	/api/v2/ai/predict	Custom ML prediction
37	System	GET	/	API home/status
38		GET	/health	Health monitoring





Step 35: Forecasting Horizons

The project specification defines three forecasting horizons.

Short-Term

- 7 days
- 14 days
- 30 days

Use cases:

- Inventory planning
- Daily pricing
- Promotion planning

Medium-Term

- 3 months
- 6 months

Use cases:

- Procurement
- Revenue forecasting
- Capacity planning

Long-Term

- 12 months

Use cases:

- Strategic planning
- Product expansion
- Annual forecasting

These horizons are defined in the PricePilot AI specification.

Important: the current XGBoost model has been rigorously evaluated on the available chronological holdout setup. It should not automatically be described as a validated 12-month forecasting model without separate long-horizon back testing.

Step 36: Demand Trend Classification

The intended forecasting system supports three demand trend categories:

- **Increasing Demand**
- **Stable Demand**
- **Decreasing Demand**

Trend classification can be derived from the slope and behavior of the forecast sequence.

This information can be displayed to business users alongside the numerical demand prediction.

Step 37: Forecast Confidence

The project specification defines a forecast confidence output between:

0% – 100%

Example:

Forecast Confidence = 88%

However, a confidence score should only be exposed as a formal calibrated confidence measure after implementing and validating an appropriate calibration method.

Therefore, confidence reporting is treated as part of the forecasting layer rather than incorrectly deriving it directly from the model's R^2 .

Step 38: Model Overfitting Analysis

The final XGBoost model produced:

Training R^2 = 0.8809

Validation R^2 = 0.8285

Test R^2 = 0.8527

Training-to-validation difference:

$0.8809 - 0.8285 = 0.0524$

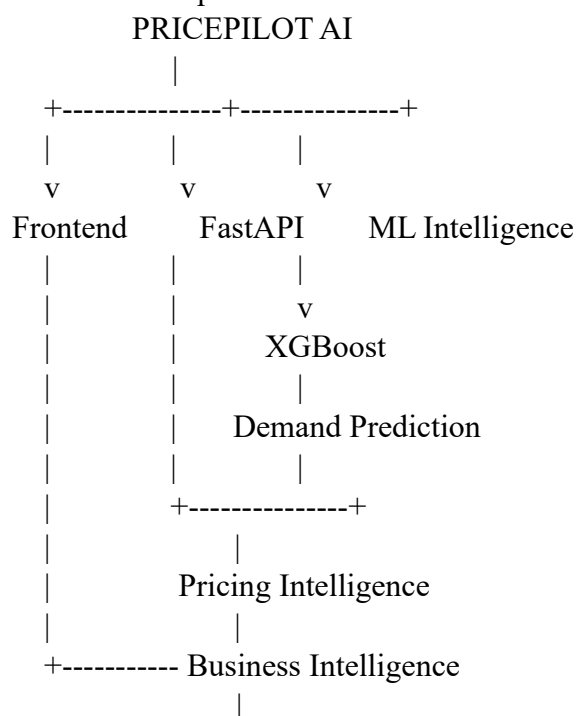
Training-to-test difference:

$0.8809 - 0.8527 = 0.0282$

The relatively small gaps indicate that the final model does not show severe overfitting under the current evaluation setup.

Step 39: Backend Database and ML Integration

The overall completed architecture can now be represented as:



The intended architecture connects PostgreSQL, the XGBoost intelligence layer and FastAPI, followed by forecasting, pricing and revenue engines and the frontend business intelligence layer.

Step 40: Technologies Used in Milestone 2

Machine Learning

- Python
- Pandas
- NumPy
- Scikit-learn
- XGBoost
- TensorFlow
- Keras

Backend

- Python
- FastAPI
- Pydantic
- SQLAlchemy
- JWT Authentication

Database

- PostgreSQL
- Alembic

Development

- VS Code
- Google Colab
- Git
- GitHub
- Postman / FastAPI Swagger

The project specification identifies FastAPI, PostgreSQL, Scikit-learn, XGBoost, Random Forest, TensorFlow, Pandas, NumPy and LSTM among the project's technology stack.

Challenges Faced

The following challenges were encountered during Milestone 2:

1. Model Selection

Different algorithms produced significantly different results. Linear models were easier to implement but could not capture the nonlinear structure of the dataset.

2. Overfitting

The initial Decision Tree and ensemble models required careful evaluation to ensure that high training performance was not simply caused by memorization.

3. Computational Cost

Hyperparameter optimization for ensemble models required careful control of:

- Number of estimators
- Tree depth
- Parallel workers
- Search space

4. LSTM Performance

The LSTM models required substantially more complexity but did not outperform the tuned XGBoost model.

5. Time-Series Feature Engineering

Lag and rolling features were tested using shifted historical values to avoid target leakage, but the experiment did not improve validation performance.

6. Backend–ML Integration

The ML pipeline must use exactly the same preprocessing and feature definitions during inference as during training.

7. Database Design

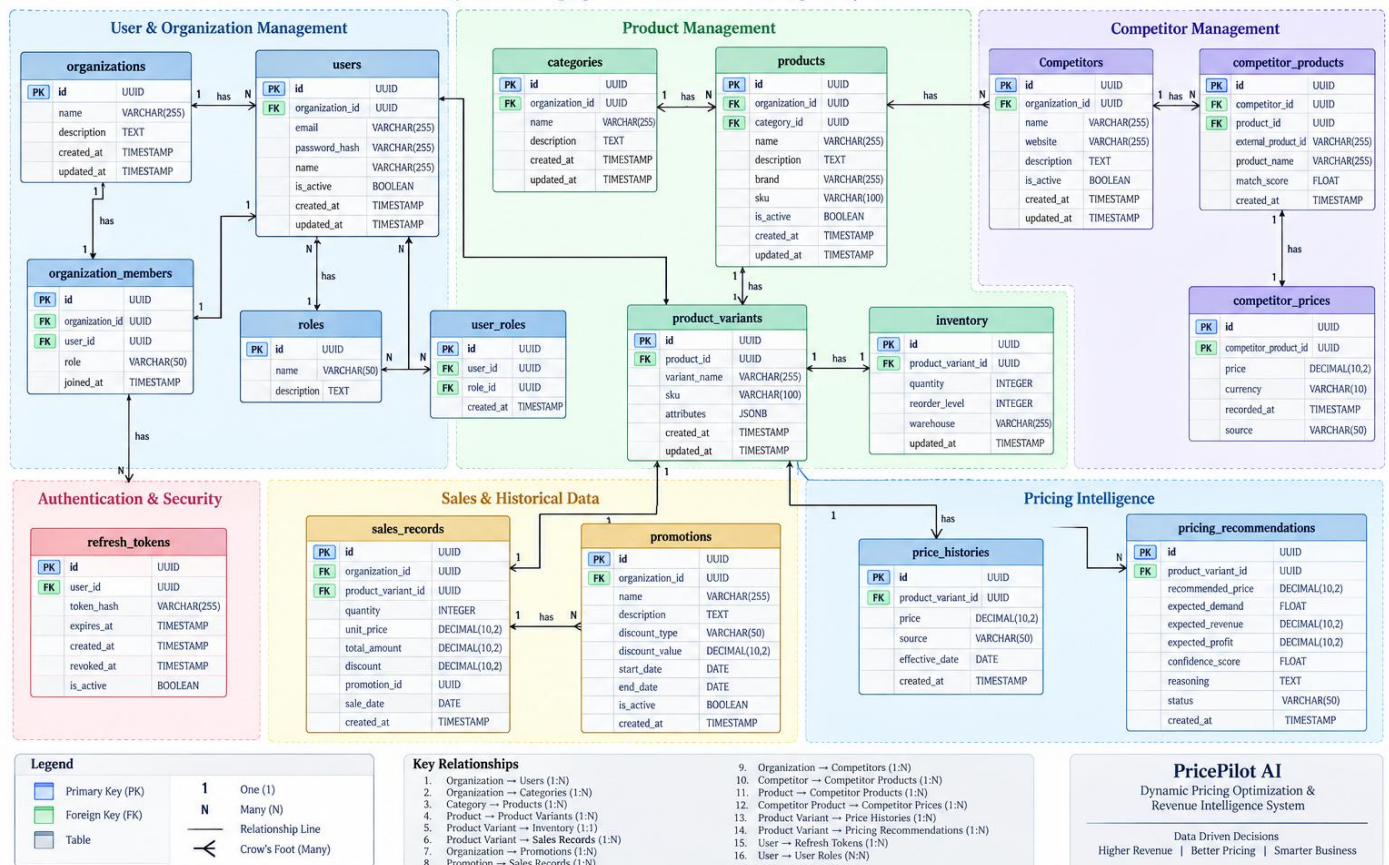
The backend required a structured relational design capable of supporting:

- Multiple organizations
- Multiple users
- Products
- Competitors
- Pricing
- Inventory
- Sales
- Recommendations

Database Schema

PricePilot AI – Database Entity Relationship Diagram

Dynamic Pricing Optimization & Revenue Intelligence System



Conclusion

Milestone 2 successfully progressed **PricePilot AI** from an exploratory data analysis project into an integrated **Machine Learning, backend and database system**.

Multiple Machine Learning algorithms were implemented and evaluated, including Linear Regression, Ridge, Lasso, Decision Tree, Random Forest, XGBoost and LSTM.

Among the tested approaches, **XGBoost demonstrated the strongest predictive performance**, achieving an **R² of 0.8527 on the held-out test dataset**, making it the current primary demand prediction model for the project.

On the software engineering side, a FastAPI backend and PostgreSQL database were established with authentication, organization management, product management, inventory, competitor, pricing, sales and AI functionality.

Submitted By

Aditya Raj Pandey

PricePilot AI: Dynamic Pricing Optimization & Revenue Intelligence System